

Networking Theory Notes

Nicoll Douglas

May 12, 2026

Contents

1	Computer Networks and The Internet	3
1.1	Packet Switching	3
1.1.1	Store-and-Forward Transmission	3
1.1.2	Packet Queueing	3
1.2	Circuit Switching	3
1.2.1	Frequency Division Multiplexing (FDM)	4
1.2.2	Time Division Multiplexing (TDM)	4
1.3	Packet Switching vs. Circuit Switching	4
1.4	Network Performance	4
1.4.1	Delay	4
1.4.2	Throughput	5
1.4.3	Factors Affecting Throughput	6
2	The Application Layer	6
2.1	HTTP	6
2.1.1	HTTP/1.1	6
2.1.2	HTTP/2	7
2.1.3	HTTP/3	7
2.2	SMTP	7
2.3	DNS	8
2.3.1	Services Provided by DNS	8
2.3.2	DNS Servers	8
2.3.3	DNS Caching	10
2.3.4	DNS Records	10
2.3.5	DNS Message Structure	11
2.3.6	DNS Zones	14
3	The Transport Layer	15
3.1	UDP	16
3.1.1	Services Provided by UDP	16
3.1.2	Segment Structure	16
3.2	TCP	17
3.2.1	Overview	17

3.2.2	Segment Structure	18
3.2.3	Head-of-Line (HOL) Blocking	20
3.2.4	Services Provided by TCP	21
3.2.5	Reliable Data Transfer	21
3.2.6	Flow Control	23
3.2.7	Congestion Control	24
3.3	TCP Connection Management	24
3.3.1	Opening a Connection	24
3.3.2	Closing a Connection	25
3.3.3	RST (Reset)	26
4	The Network Layer: Data Plane	26
4.1	Overview	26
4.1.1	Data and Control Planes	26
4.1.2	Forwarding and Routing	26
4.1.3	Network Service Model	27
4.2	Routers	27
4.2.1	Components	27
4.2.2	Input Port Processing	28
4.2.3	Switching	29
4.2.4	Queuing	30
4.2.5	Packet Scheduling	32
5	The Data Link Layer	32
5.1	Frames	32
5.1.1	Structure	33
5.1.2	Data Loss	34
5.1.3	Sending Frames	34
5.1.4	Building Ethernet Frames	35
5.2	Link Layer Switches	35
5.2.1	Forwarding	35
6	The Physical Layer	36
6.1	Access Networks	36
6.1.1	Digital Subscriber Line	36
6.1.2	Cable Internet	36
6.1.3	Other Forms of Access	36
6.2	Physical Media	37
	Glossary	37

1 Computer Networks and The Internet

1.1 Packet Switching

- [Packet switching](#) is a method of transmitting data through a network by breaking it up into small units called packets.
- Each packet is sent independently through the network and may take different paths to reach the destination, where the packets are reassembled into the original message.
- Network application exchange messages with each other which are broken down into packets at the source.
- Between source and destination, packets travel through communication links and [packet switches](#) (routers and link-layer switches mainly).
- Packets are transmitted over each link at a rate equal to the full [transmission rate](#) of the link.

1.1.1 Store-and-Forward Transmission

- [Store-and-forward](#) transmission means that packet switches must receive an entire packet before it can transmit the first bit of the packet onto the outbound link.
- Packet switches do this because they need to process data in the packet in order to forward it to the correct place.

1.1.2 Packet Queueing

- Each outbound link of a packet switch has an output buffer (queue) which stores packets the switch will send out of that link.
- If a packet arrives at the queue but there are other packets in front, it will suffer a queueing delay.
- If an output buffer is full, packet loss will occur as any incoming packets will be dropped.

1.2 Circuit Switching

- [Circuit switching](#) is a method of transmitting data through a network by creating a dedicated path or circuit from one endpoint to another for the entire duration of a session, ensuring a constant and reserved channel for data transmission.
- When a network establishes a circuit, it also reserves a constant transmission rate in the network's links (representing a fraction of each link's transmission capacity).

- Since a fixed transmission rate has been reserved, the sender can transfer data to the receiver at the guaranteed constant rate.

1.2.1 Frequency Division Multiplexing (FDM)

- With [Frequency Division Multiplexing \(FDM\)](#), the frequency spectrum of a link is divided up among the connections established across the link.
- The frequency band for telephone networks is 4kHz.
- The width of the band is called the bandwidth
- FM radio stations also use FDM to share the frequency spectrum between 88MHz and 108MHz.

1.2.2 Time Division Multiplexing (TDM)

- With FDM the frequency domain is split up however with [Time Division Multiplexing \(TDM\)](#), the time domain is split up.
- Time is split into frames, each lasting for T bits.
- If a link has N connections then each connection gets N/T bits of transmission every T bits (rotates periodically across connections).

1.3 Packet Switching vs. Circuit Switching

- An argument against packet switching is that it is not suitable for real-time services (telephone calls, video conferencing) because of unpredictable end-to-end delays (due primarily to queueing delays).
- An argument against circuit switching is that it offers better sharing of transmission capacity.
- Packet switching is less costly to implement than circuit switching.
- During silent periods where no data is transmitted, bandwidth is wasted with circuit switching.
- [Packet switching](#) allocates link use on demand whereas circuit switching pre-allocates bandwidth.

1.4 Network Performance

1.4.1 Delay

- When a packet has fully arrived at a packet switch, it may experience several delays before arriving at the next packet switch.
- Other than the ones mentioned below, end systems can also produce delays for example in packetization and depacketization of messages.

Processing Delay

- The time required to examine the packet's header and determine where send it forms part of the [processing delay](#).
- The processing delay may also include the time it takes for encapsulation and decapsulation of packet layer information.
- Devices processing the packet may also verify packet checksums to detect bit errors.

Queueing Delay

- A packet experiences a [queueing delay](#) if it is ready to be transmitted onto a link but it has to wait (either there is a packet already in transmission or the queue is non-empty).
- Queueing delays can be measured using the traffic intensity.
- The traffic intensity is the ratio of the average rate at which bits arrive at a link to the transmission rate that they are pushed out of the link.
- A traffic intensity > 1 means that the packet queue will grow, whereas ≤ 1 means the queue is stable or will shrink.

Transmission Delay

- The [transmission delay](#) is the amount of time required to push all bits of a packet into a link.

Propagation Delay

- The [propagation delay](#) is the amount of time required for a bit to travel across a link.
- The speed at which the bit travels depends on the physical medium and thus so does the delay.

1.4.2 Throughput

- [Instantaneous throughput](#) at any instant of time is the rate (*bits/sec*) at which a receiver is receiving bits from a sender that form part of a piece of data.
- [Average throughput](#) measures the average rate at which a receiver is receiving bits from a sender that form part of a piece of data (the quotient of the total bits and total data transfer time).

1.4.3 Factors Affecting Throughput

- From end-to-end, there are several links connecting packet switches.
- One of these links may be a global bottleneck link i.e the link with the lowest transmission rate.
- The [throughput](#) thus depends on the transmission rate of the bottleneck link as data cannot flow faster end-to-end than the bottleneck allows.
- Throughput also depends on queueing delays. Example:
 - 10 unrelated pieces of data / messages are being transmitted across a link.
 - This means packets will get queued and the link has to share its capacity.
 - Assuming fair sharing of packets, the effective transmission rate of the link is now divided by 10 and so is the throughput across each message.

2 The Application Layer

2.1 HTTP

2.1.1 HTTP/1.1

Without Pipelining

- Only one HTTP request can be handed off to a TCP socket / be in-flight at a time.
- The application must wait for the response before sending off the next request.
- Requests behave like a FIFO queue and [head-of-line blocking \(HOL\)](#) occurs because subsequent requests must wait for the earlier requests to return before they can be sent off.

With Pipelining

- [HTTP/1.1 pipelining](#) allows multiple HTTP requests to be handed off to a TCP socket / be in-flight at a time.
- Multiple responses can't be in-flight, only one at a time.
- On the server-side, responses are queued according to which requests arrive first (may not be the same order in which they were sent).
- HOL still occurs on the response side because even if other responses are ready to go, they must wait for the current in-flight response to be delivered.

Workarounds

- Modern browsers do not use pipelining and instead open multiple TCP connections per origin (6-8) which is more efficient.
- Requests are then spread across them.
- This helps but also wastes resources, increases congestion, adds TCP overhead, and only reduces latency rather than eliminating HOL.

2.1.2 HTTP/2

- HTTP/2 assigns request/response pairs a stream and an ID.
- It also breaks down its messages into frames at the application layer.
- Each frame is given the stream ID for the respective request/response.
- HTTP/2 takes frames from multiple streams and passes them to a TCP socket, interleaving them based on priority and flow control.
- If packet loss occurs, HOL still occurs at the transport layer.
- Any buffered TCP segments have to wait for the earlier lost segment to be retransmitted.
- Any buffered, contiguous TCP segments that build up into frames ready for processing by HTTP/2 at the application layer also have to wait (this is blocking still at the transport-level, not application level).

2.1.3 HTTP/3

- HTTP/3 eliminates transport-layer blocking entirely by using the QUIC protocol instead of TCP.

2.2 SMTP

- A user agent allows a user to read, reply to, forward, save and compose e-mails.
- When a user sends a message from their user agent, the user agent sends it over to the user's mail server.
- The outgoing e-mail is placed in an outgoing message queue.
- The mail server sees it in the message queue and sends it over to the recipient's mail server via SMTP (which uses TCP).
- The recipient's mail server receives the e-mail and places it in the recipient's mailbox.
- The recipient can then read the e-mail at their convenience.

- If more than one e-mail in the message queue goes to the same server, the client sends them over the same TCP connection.

2.3 DNS

- The Domain Name System (DNS) is a distributed database implemented in a hierarchy of DNS servers.
- It is also an application-layer protocol that allows hosts to query the distributed database for hostnames that translate to IP addresses.
- End systems usually have DNS applications that perform the client-side of DNS (querying).

2.3.1 Services Provided by DNS

Host Aliasing

- A hostname can also have one or more alias names that map to it (aliases).
- The underlying hostname that maps to the IP is known as the canonical hostname.

Load Distribution

- Web servers can have multiple replicas with each server running on a different end system.
- For replica web servers, a set of IP addresses is associated with a hostname.
- DNS queries for the hostname are replied with the entire set of IP addresses but the ordering is rotated.
- Clients typically use the IP address at the top of the list so rotation provides load distribution.

2.3.2 DNS Servers

- DNS queries can be *recursive* or *iterative*.
- When a requesting host sends a query Q to a recursive resolver to resolve on their behalf, query Q is a [recursive DNS query](#).
- When a requesting host or recursive resolver makes a direct DNS query (say to a root server or TLD server) that forms part of a larger query Q , each subquery is an [iterative DNS query](#) as they form part of a larger query and each gets returned to the requesting host.
- In practice, the path a DNS query takes follows the points given below.

Root DNS Servers

- There are 1000+ root server instances across the world that are copies of 13 different root servers.
- The 13 servers are managed by 12 different organisations and coordinated by the Internet Assigned Numbers Authority (IANA).
- Root servers provide the IP addresses of TLD servers.
- When a requesting host makes a query for a hostname, it will first contact a root server (an iterative query).

TLD Servers

- For each top-level domain, there is a TLD server (or server cluster).
- TLD servers provide the IP addresses for authoritative DNS servers.
- When a client gets the IP address of a TLD server from a root server, it can then contact the TLD server for an authoritative name server (an iterative query).
- TLD servers may not always know the authoritative name server and only an intermediate name server which knows the authoritative server (zone delegation).

Authoritative DNS Servers

- Authoritative DNS servers house DNS records that map names of publicly accessible hosts to IP addresses.
- The owner of the hosts often pays service providers to have these records stored in their authoritative name servers (think Namecheap, Cloudflare, etc.).
- When a requesting host gets the IP address of an authoritative name server from a TLD server, it can then contact the authoritative name server for the IP address of the hostname (iterative query).

Local DNS Servers

- ISPs have local DNS servers.
- When a host connects to an ISP, the ISP provides the host with the IP addresses of one or more of its local DNS servers that is close to the host.
- When hosts make DNS queries, they are typically sent to the local DNS server which acts as a proxy forwarding the query into the aforementioned hierarchy of root $\Rightarrow$ TLD $\Rightarrow$ authoritative.

- There also exist public DNS servers such as Google's (8.8.8.8) or Cloudflare's (1.1.1.1) which perform the same function.
- These types of servers are known as recursive resolvers, performing DNS queries on behalf of a requesting host.

2.3.3 DNS Caching

- When a resolver receives a reply to an iterative DNS query it caches the result locally.
- Resolvers discard cached DNS records after a certain time known as the Time-To-Live (TTL) set in the record itself.
- Common TTLs are 5mins, 1hr and 24hrs which mix flexibility and speed at varying levels.
- DNS record propagation means waiting for various caches around the world to expire with old records so the authoritative name server is queried again.
- End systems also maintain caches at the OS level (system-wide cache) as well as in web browsers or other applications that accesses the internet.
- When changing nameservers of a DNS record, records are being changed at the TLD level.
- These TLD servers often have a high non-configurable TTL (usually 48 hours) so any incoming queries will use cached records from the TLD instead of hitting the new nameserver.

2.3.4 DNS Records

- DNS servers store resource records (RRs).
- A resource record contains 4 fields: name, value, type and TTL.
- MX records allows a company to point a hostname to another hostname of a mail server that handles mail for the original hostname.
- If a DNS server is authoritative it will contain A records for the hostname.
- If a server is not authoritative it will contain NS records for the domain that includes the hostname (a delegation).
- It may also contain an A record that provides the IP address of that nameserver (glue).
- Registrars enter DNS records into the DNS database.
- When registering a domain with a registrar, you have to provide the authoritative name servers (your provider may also provide them for you).

Type	Name	Value
A	A hostname for an IP address	The IP address
NS	A hostname	The hostname of an authoritative name server that can resolve the domain
CNAME	An alias hostname	The canonical hostname for the alias hostname
MX	An alias hostname	The canonical hostname for the alias hostname of a mail server

Table 1: The name and type values for different types of DNS records.

- The registrar will then propagate an NS record and an A record into all of the servers for the respective TLD.

2.3.5 DNS Message Structure

- DNS messages have the same structure for queries and replies.
- DNS messages use big endian byte ordering.
- DNS messages usually fit inside one UDP packet.
- DNS uses UDP because it is quicker and lighter.
- If there's no response to a DNS query the client just retries.
- If the response is too big or truncated, the server sets the TC flag and the client retries using TCP.

Header

- Size: 12 bytes

ID

- Size: 2 bytes
- The first 2 bytes of the header is a 16-bit number that identifies the query and reply (same ID in each pair)

Flags

- Size: 2 bytes

Flag Name	Length	Description
QR	1 bit	Query/Response: 0 = query, 1 = response
Opcode	4 bits	Operation code: 0 = Standard query, other codes
AA	1 bit	Authoritative Answer (response only): Server is the authoritative server of a hostname
TC	1 bit	Truncated: 1 = Message was too large for UDP and was cut off
RD	1 bit	Recursion Desired: The server should perform recursive resolution
RA	1 bit	Recursion Available (response only): Server supports recursion
Z	3 bits	Reserved: Must be 0, reserved for DNSSEC and future features
RCODE	4 bits	Response Code: 0 = No error, 3 = Name error (NXDOMAIN), other codes

QDCOUNT

- Size: 2 bytes
- The number of entries in the question section of a query.
- Is a 16-bit unsigned integer.
- Responses usually copy this value over from the query.

ANCOUNT

- Size: 2 bytes
- The number of resource records in the answers section.
- Is a 16-bit unsigned integer (0 in a query).

NSCOUNT

- Size: 2 bytes
- The number of resource records in the authority section.
- Is a 16-bit unsigned integer (0 in a query).

Additional Record Count

- Size: 2 bytes
- The number of resource records in the additional section.
- Is a 16-bit unsigned integer (0 in a query).

Question Section

- There are QDCOUNT questions.
- Each question is formed by a QNAME, a QTYPE and a QCLASS.
- The QNAME of a question is a sequence of labels, each label is a sequence of 1-byte ASCII characters prefixed by a length byte indicating the length of the label (characters only).
- The QNAME is terminated by a zero-length label (0x00).
- The QTYPE is a 16-bit unsigned integer and represents what record is being asked for (A=1, AAAA=28, MX=15, etc.).
- The QCLASS is a 16-bit unsigned integer, usually 1=IN (Internet).

Resource Record Sections

- All other sections use the same format (answer section, authority section, additional sections).
- They are composed of the resource records that the server responds with (each record includes the name, type, value and TTL of the record).

Common Response Scenarios

Successful Lookup

- Server is authoritative for the name and record type.
- Answer section: Contains the requested RRs.
- Additional section: May contain optional helper RRs.

CNAME

- Queried name is an alias and points to another name that must be resolved.
- Answer section: CNAME record (and possible final RR).
- Additional section: Optional helper RRs.

Delegation (referral)

- Server is not authoritative and tells resolver where to ask next.
- Authority section: NS records for child zone
- Additional section: Potential glue records for NS.

NXDOMAIN

- Queried name doesn't exist in the authoritative zone.
- Authority section: SOA of authoritative zone.

NODATA

- The name exists in the authoritative zone but the requested record type doesn't.
- Authority section: SOA of authoritative zone.

2.3.6 DNS Zones

- A **DNS zone** is a portion of the DNS namespace that a DNS server is authoritative for.
- It contains authoritative A records for the domains it manages.
- It can also contain delegation records (NS records) for child zones.
- A zone file is a file stored at the authoritative server that stores all DNS records for the zone (acting as the database).
- The apex is the topmost domain name in a DNS zone.

Zone Delegation

- A DNS server *A* can delegate a portion of its namespace to another DNS server *B*, creating a zone.
- This is done by creating an NS record in *A* to point to another authoritative server *B* for one of *A*'s domains (e.g sub.example.com).
- Now server *B* is the authority for the domain and its subdomains.
- In order to remove circular dependency for this record, a glue record (A type) may be added to *A*.
- Glue records are not authoritative and the authoritative source is the child zone authoritative server (*B*).

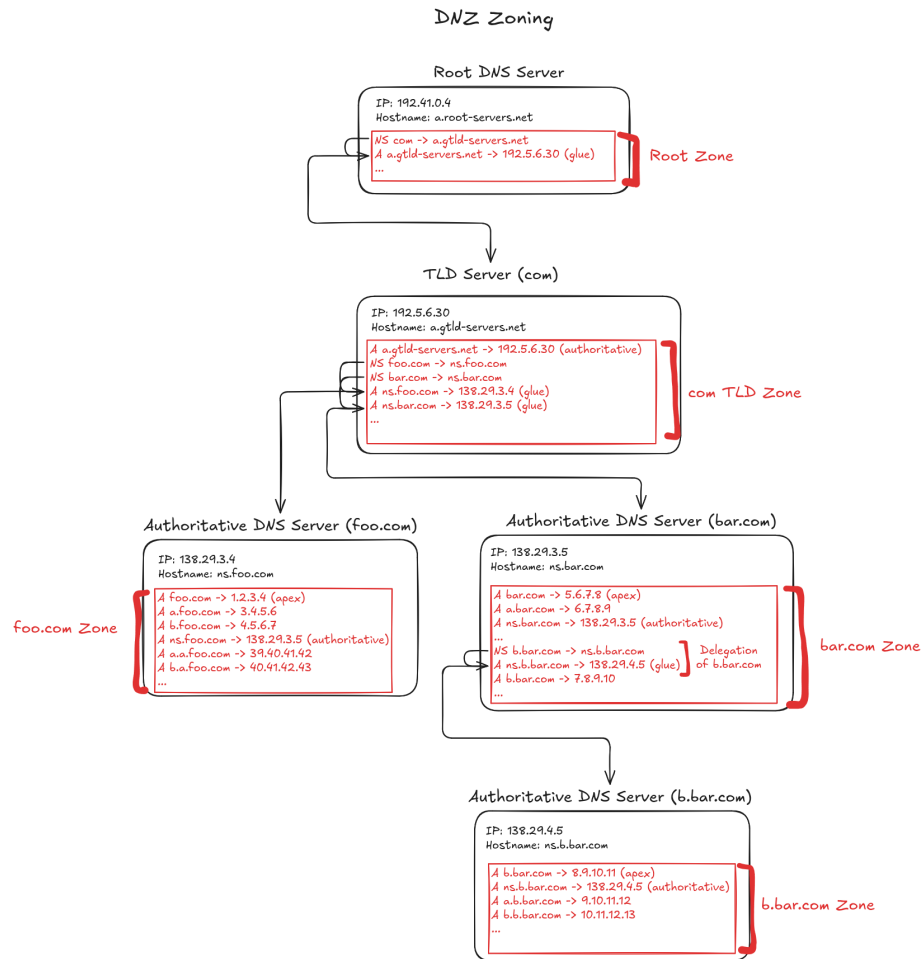


Figure 1: Example of DNS Zoning

SOA (Start of Authority) Record

- Every DNS zone has exactly one SOA record located at the authoritative server for that zone.
- This record tells who is authoritative for the zone, how secondary servers keep the zone in sync, and how long to cache negative answers.

3 The Transport Layer

- Transport-layer protocols provide logical communication between *processes* running on different hosts.

- In comparison, network-layer protocols provide logical communication between *hosts*.

3.1 UDP

3.1.1 Services Provided by UDP

- Process-to-process data delivery.
- Segment integrity checking by including error-detection fields in the segment's header.
- Multiplexing/demultiplexing is as simple as specifying the host and destination ports in the headers of a UDP segment.
- When a sender specifies the source port it acts as a "return address".
- UDP does not provide flow control and if receive buffer at the receiver overflows (segments arrive faster than the application reads) then packet loss occurs at layer 4.

3.1.2 Segment Structure

Source Port

- Size: 16 bits
- The source port numbered 0 to 65535.

Destination Port

- Size: 16 bits
- The destination port numbered 0 to 65535.

Length

- Size: 16 bits
- The total length of the segment (header + data) in bytes.
- Minimum value is 8 bytes (just the header).
- Theoretical maximum is 65535 bytes but usually limited by the MTU of the network.

Checksum

- Size: 16 bits
- The checksum which is used for error-checking the header and data to ensure integrity.
- The sender calculates a checksum over the segment's bits.
- The receiver does the same maths, if the numbers don't match, the segment was corrupted in transit and typically discarded.

Payload

- Size: Theoretical maximum of 65527 (65535 - 8 header bytes).
- The packet payload.

3.2 TCP

3.2.1 Overview

Multiplexing and Demultiplexing

- Refers to extending the host-to-host delivery service to a process-to-process delivery service for applications running on the hosts.
- The transport-layer in the receiving host doesn't deliver data directly to the process but to an intermediary socket (the OS's networking API).
- Each transport-layer segment has a set of fields in the segment for this purpose.
- Multiplexing and demultiplexing also happen at other layers of the OSI model whenever a protocol in the layer is used by other protocols higher up.

Demultiplexing

- In a receiving host, the transport layer examines the fields a transport-layer segment in order to identify the receiving socket and then directs the segment to that socket.
- This job of delivering the data in a transport-layer segment to the correct socket is called demultiplexing.

Multiplexing

- The job of gathering data chunks passed to the transport-layer at the source host, encapsulating each chunk with header information to create segments, and passing the segments to the network layer is called multiplexing.

Sockets and Port Numbers

- Sockets on end hosts must be uniquely identified in order for multiplexing and demultiplexing to work at the transport-level.
- The unique identifiers used are the source port number field and the destination port number field in a transport-layer segment.
- Each port is an unsigned 16-bit number (0 to 65535).
- Port numbers ranging from 0 to 1023 are well-known port numbers and are restricted.

3.2.2 Segment Structure

- MTU (Maximum Transmission Unit) is the largest physical packet size that a network interface (e.g ethernet cable) can handle.
- The standard size for ethernet networks is 1500 bytes which is the size of an Ethernet frame's payload (an IP packet).
- MSS (Maximum Segment Size) is the largest amount of application data that TCP can handle in a single segment.
- MSS is usually 1460 bytes since the IP header is 20 bytes and the TCP header is 20 bytes.

Source Port (16 bits)

- Size: 16 bits
- The source port (0 to 65535).

2. Destination Port (16 bits)

- Size: 16 bits
- The destination port (0 to 65535).

Sequence Number

- Size: 32 bits
- The sequence number for a segment is calculated from the byte stream given to TCP.
- E.g, if the byte stream consists of 500,000 bytes and the MSS is 1,000 bytes, TCP will construct 500 segments.
- The bytes will be numbered from 0 to 499,999 and the sequence number will be the byte's number for where the segment starts (0, 1000, 2000, etc.).

- Sequence numbers increase by the payload size sent in that packet (so if current seq. is 0 and you send a packet of 1000 bytes, current seq. becomes 1000).
- Both sides choose a random initial sequence number which is done to minimize the possibility that a segment still present in the network from an earlier already-terminated connection is mistaken for a valid segment in a later connection.
- Sequence numbers range from 0 to about 4 billion (4GB); when the max is reached the sequence number wraps around back to 0.

Acknowledgement Number

- Size: 32 bits
- The number of the next byte the receiver expects to receive.
- If a receiver sends an ACK with the number 1001, it is saying "I have successfully received everything up to and including byte 1000, please send 1001 next".
- E.g, if a sender sends a segment with sequence number 500 and data length 200 (payload), the receiver will send an ack. number of 700 (we got bytes 500-699 now give us 700+).
- The ack. number is cumulative so it can only advance if all previous bytes have been received.
- E.g, seg A = bytes 1-100, seg B = bytes 101-200, seg C = bytes 201-300.
 - Seg A arrives at receiver, receiver ACKs seg A by using ack. number 101 (asking for seg B).
 - Seg C arrives at receiver, receiver sees out of order packet, buffers it but ACKs 101 (asking again for 101 because it cannot ACK seg C yet).
 - The sender receives a duplicate ACK and realizes 101 is missing at the receiver and retransmits it.
- Acknowledgement numbers increase by the number of bytes received in an in-order payload from the other side.
- ACKs often piggyback on data being sent if the client and server are both exchanging data simultaneously.

Header Length

- Size: 4 bytes
- Tells the receiver the length of the header in 32-bit words.
- A TCP header can have variable length due to the options field however it is usually empty making the header 20 bytes and the length field 0101 (5 32-bit words e.g 5 x 4 bytes = 20).

Flags

- Size: 9 bits
- Flags that act as control bits to manage the state of the connection.
- SYN: synchronize (start a connection).
- ACK: indicates the acknowledgment field is valid (the segment contains an acknowledgement for a segment that has been successfully received).
- FIN: finished (close a connection).
- RST: reset the connection.

Receive Window

- Used for flow control, tells the sender "I only have room for X more bytes".

Checksum

- Used for error detection.

3.2.3 Head-of-Line (HOL) Blocking

- A TCP connection is a reliable, in-order, bidirectional stream of bytes.
- When an application passes a byte stream to a TCP socket, it groups them in-order into segments.
- The receiving side buffers incoming segments and reorders them if they arrive out of order.
- The TCP socket on the receiving side will not deliver bytes to the application until all earlier bytes are present.
- TCP will only deliver a contiguous prefix of the byte stream to the application. Example:
 - A receiving side receives segments 2, 3, and 4.
 - It notices segment 1 is missing.

- Then, it will request retransmission of segment 1 and won't deliver 2, 3, and 4 to the application.
- Even though most data has arrived, TCP will wait and buffer it until all previous data has arrived.
- This is transport-layer head-of-line blocking.

3.2.4 Services Provided by TCP

1. Reliable data transfer ensuring data is correct and in order.
2. Flow control (regulating the rate at which data is transferred to help the receiver not be overloaded).
3. Congestion control (a service provided not so much to the application but to the internet as a whole).
4. Process-to-process data delivery.

3.2.5 Reliable Data Transfer

- Pipelining is used to avoid poor performance of stop-and-wait protocols and improve the utilization of the sender.
- The sender can start sending any data it has whilst some of it is already in transit instead of waiting.
- Pipelining allows the sender to send multiple packets without waiting for acknowledgements.
- The introduction of pipelining has the following consequences:
 - The range of sequence numbers must be increased since each in-transit packet must have a unique sequence number.
 - The sender and receiver sides may have to buffer more than one packet (at minimum the sender has to buffer packets that have not yet been ack'd).
 - The range of seq. numbers and buffering requirements will depend on how the protocol responds to lost, corrupted or overly delayed packets.
 - * 2 basic approaches to pipelined error recovery can be identified: Go-Back-N and Selective Repeat.

TCP Implementation

- TCP uses a single retransmission timer.
- When data is passed to TCP, a timer is started if there isn't already one running.

- The timeout is associated with the oldest unacknowledged packet.
- When the timeout occurs, TCP retransmits the oldest unacknowledged segment with the smallest sequence number.
- When TCP retransmits it doubles the timeout interval (from what the previous value was).
- The timer expiration is usually caused by congestion in the network and in times of congestion if the sender continues to retransmit packets persistently the congestion may get worse.
- TCP uses cumulative acknowledgements so when a new ack. number is received, the value of the sequence number of the oldest unacknowledged byte is updated and the timer restarted (assuming its bigger than what was known).
- Timeout-triggered retransmissions increase end-to-end delay due to the sender withholding the segment.
- TCP functions like Go-Back-N but instead of resending all unacknowledged packets on timeout it only retransmits the oldest unacknowledged packet.
- TCP also relies on duplicate ACKs, if three duplicate ACKs are received then the oldest unacknowledged packet is retransmitted immediately instead of waiting for the timeout (fast retransmit).

Round-Trip Time Estimation and Timeout

- A new value for the sample RTT can be obtained once every RTT (since we get a new approximation / data point).
- The sample RTT fluctuate between segments due to congestion in routers.
- TCP calculates its timeout using an estimate of the round trip time plus some margin.
- The estimate is calculated using an average such that newer samples of the RTT provide more weight in the average.
- The margin is calculated based on how much the RTT samples vary; when there is a lot of fluctuation, the margin is large, when there isn't any, the margin is small.
- The initial timeout interval is 1 second.

3.2.6 Flow Control

- TCP senders and receivers each have a send buffer and a receive buffer.
- The send buffer contains data that was sent but not yet acknowledged and kept for the case of retransmission as well as data that TCP is waiting to transmit (waiting for the window to slide up).
- The receive buffer contains in-order data that was received and that is ready for the application above to read.
- TCP has a flow control service that it provides to its application to prevent the possibility of the sender overflowing the receiver's receive buffer.
- Flow control acts like a speed-matching service that matches the rate at which the sender is sending against the rate at which the receiving application is reading.
- The sender maintains a "receive window" state variable which gives the sender an idea of much free buffer space is available at the receiver.
- The LastByteRead variable is the number of the last byte read from the buffer by the application process.
- The LastByteRcvd variable is the number of the last byte received and placed in the buffer.
- $rwnd = RcvBuffer - (LastByteRcvd - LastByteRead)$; the receive window is the amount of remaining space available in the buffer.
- The rwnd value is placed in the receive window field of the header of every segment that is sent back from the receiver.
- The sender on the other side maintains the LastByteSent and LastByteAcked variables (the difference between them is the amount of unacknowledged data sent into the connection).
- By keeping the amount of unacknowledged data less than rwnd, the sender prevents the receive buffer from overflowing on the other side.
- However, if rwnd is 0 and the sender has no more data to send then no data segments or ACKs will be transmitted which stops any future data received from above from being transmitted when space eventually opens up in the receive buffer and the sender has data to send.
- TCP uses a mechanism call Zero Window Probe (ZWP) which starts a persist timer that runs indefinitely when rwnd is 0 to prevent this.
- The sender will send the next byte of data in the sequence after the timer expires (intentionally breaking the rules) to force a response from the receiver with a potential new rwnd value.

- If the window is still 0, the persist timer is restarted and doubles the wait time.

3.2.7 Congestion Control

Costs of Congestion

- As packet arrival rate nears link capacity (congestion), large queueing delays are experienced.
- As packet loss occurs (due to congestion), the sender has to perform re-transmissions to compensate for lost packets.
- If a network is congested and experiencing large queueing delays, the sender may timeout and retransmit the packet when it is unneeded (e.g. previous packet received or still in transit), wasting link bandwidth that could've been better spent on other packets.
- When a packet is dropped at a router, the transmission capacity used for forwarding the packet at each link between that router and the sender was wasted.

TCP Implementation

3.3 TCP Connection Management

3.3.1 Opening a Connection

Step 1 - Send SYN

- The client sends a data-less TCP segment with the SYN flag set to 1 in the header.
- The client also adds the randomly chosen initial sequence number to the header.

Step 2 - Receive SYN, Send SYNACK

- The receiver receives the SYN segment and allocates the TCP buffers and variables to the connection.
- A connection-granted (SYNACK) segment is then sent to the client.
- The SYN bit here is set to 1.
- The ACK number here is set to the client's initial sequence number + 1.
- The server also adds its randomly chosen initial sequence number to the header.

Step 3 - Receive SYNACK

- When the client receives the SYNACK segment, it also allocates variables and buffers to the connection.
- The client then sends another segment to the server with the ACK number set to the server's initial sequence number + 1 (acknowledging the connection-granted SYNACK segment).
- Here the SYN bit is set to 0 since the connection is established.
- This segment may contain application data in the payload.

3.3.2 Closing a Connection

- Because TCP is full-duplex, each direction of the connection must be closed independently.

Step 1 - Client FIN

- The client sends a segment with the FIN flag set to 1 (when the client has no more data to send).
- The client then enters the FIN-WAIT-1 state (waiting for the ACK).

Step 2 - Server FINACK

- The server then responds with an ACK acknowledging that the client wants to close the connection.
- The server then enters the CLOSE-WAIT state and the client enters the FIN-WAIT-2 state.

Step 3 - Server FIN

- The server sends a segment with the FIN flag set to 1 (when the server has no more data to send).
- The server then enters the LAST-ACK state.

Step 4 - Client FINACK

- The client then responds with an ACK acknowledging that the server wants to close the connection.
- The client then enters a TIME-WAIT state (safety buffer).
- The server receives this ACK and immediately closes the connection (enters the CLOSED state).

- In the TIME-WAIT state the client enters a wait period (usually twice the maximum segment lifetime).
- This is done for reliability i.e if the client's FINACK gets lost there is a window for the server to resend its FIN and the ACK to be resent.
- This also allows wandering packets from the old connection to die off in the network, preventing a new connection using the same port numbers from accidentally receiving a delayed packet from the old connection.
- If the last ACK has to be retransmitted the TIME-WAIT timer is restarted.

3.3.3 RST (Reset)

- If a server host receives a TCP SYN segment but the host is not listening for TCP connections on the target port, the host will then send a segment with the RST flag set to 1.
- It tells the sender that it is not accepting TCP connections on that port.

4 The Network Layer: Data Plane

4.1 Overview

4.1.1 Data and Control Planes

- The network layer can be subdivided into two planes:
 - **Data plane:** Functions that determine how a datagram arriving on a router's input link is forwarded to an output link
 - **Control Plane:** Network-wide logic that controls how a datagram is routed amongst routers on an end-to-end path from source to destination

4.1.2 Forwarding and Routing

- The job of the network layer is to move packets from a sending host to a receiving host.
- Two network functions are required for this:
 - **Forwarding:** A router deciding which output link an incoming packet should go to.
 - **Routing:** The network deciding the route or path taken by packets flowing from sender to receiver (algorithms that calculate these paths are called *routing algorithms*).
- A key aspect of a router is its *forwarding table*.

- The router looks at the headers in a packet and then uses these values to index its forwarding table.
- The value stored for a forwarding table entry indicates the outgoing link that correspond to those header values for the packet.
- The contents of a forwarding table are determined by a routing algorithm.
- The routing algorithm function in one router and another communicate via a *routing protocol* to compute the values for its forwarding table (traditional control plane approach).
- Routing can also be *software-defined* where an external remote controller performs control-plane functionality for a network instead of router-to-router.
- These are known as *software-defined networks* (SDNs).

4.1.3 Network Service Model

- The network service model defines the characteristics of end-to-end delivery of packets between senders and receivers.
- The internet's network layer provides a single service known as best-effort service where no guarantees are made (e.g on delivery, delay, in-order delivery, etc).

4.2 Routers

4.2.1 Components

- Four components of a high-level, generic router can be identified.
- A router's input ports, output ports, and switching fabric are almost always implemented in hardware.
- So the data plane is also hardware-level, operating at the nanosecond time scale whereas control-plane functionality operates at millisecond or second timescale in software.

Input Ports

- Performs the physical layer function of terminating an incoming physical link.
- Performs link-layer functions needed to interoperate with the link layer at the other side of the link (e.g another router's output port).
- A lookup function is also performed at the input port which consults the forwarding table to determine the necessary output port for a packet received.

- Afterwards, the port will pass on the packet to the output port via the switching fabric.
- A control packet (e.g packets carrying routing protocol information) are forwarded to the routing processor instead of an output port.

Switching Fabric

- The switching fabric connects the router's input ports to its output ports.

Output Ports

- Stores packets received from the switching fabric.
- Transmits them onto the outgoing link by performing necessary link-layer and physical-layer functions.
- When a link is bidirectional (carries traffic in both directions), components of an output port will be paired with components of an input port in one unit (the same line card).

Routing Processor

- Performs control-plane functions.
- In traditional routers it executes routing algorithms, maintains routing tables and attached link state information, as well as computing the forwarding table.
- In SDN routers, the routing processor is responsible for communicating with the remote controller in order to receive forwarding table entries computed externally.
- The routing processor also performs network management functions.

4.2.2 Input Port Processing

- Input port processing and destination-based forwarding consist of the following functions at input ports.
- Other aspects of processing like blocking, queueing, and scheduling of packets at both entry and exit of the input port are also managed at input ports.

Physical-Layer Processing

- The input port must demodulate the physical signals of a packet into a digital representation i.e the layer 2 packet in the input port memory.

Link Layer Processing

- The router performs a CRC for the FCS in the link-layer frame to check data integrity.
- The router checks that the MAC address is for the intended router.
- The link-layer frame must also be decapsulated into an IP packet.

IP Processing

- The IP packet's checksum is verified.
- The destination (IP) is identified in order to perform lookup.

Lookup

- The forwarding table in a router is either computed with a routing protocol or received from a remote SDN controller.
- The forwarding table (at the routing processor) is copied to the input line cards over a separate bus.
- This allows for local lookup and forwarding decisions instead of invoking the central processor which could cause a bottleneck.
- Instead of a forwarding table holding, say, 4 billion IP addresses, it can have ranges of IP addresses corresponding to each of its outgoing links.
- This involves using a prefix to match a range of IP addresses with the same prefix.
- When there are multiple matching prefixes, the router uses the *longest prefix matching rule*.
- The router uses the forwarding table to look up which output port the IP packet must be forwarded to.

4.2.3 Switching

Switching via memory

- Switching is controlled by the CPU of the router.
- The packet is copied from the input port into the processor memory, a forwarding decision is made, and then the packet copied to the necessary output port's buffers.
- If the memory bandwidth of the system bus is B packets per second, then the total switching throughput is $\leq B/2$ since every packet must cross the system bus twice (once to enter process memory and once to leave).

Switching via a bus

- An input port transfers a packet directly to the output port via a shared bus (without intervention from the routing processor).
- Typically done by having an input port attach a switch-internal label indicating the output port.
- All output ports attached to the bus receive the packet but only the port that matches the label keeps it.
- The label is then removed at the output port (no longer needed).
- Only one packet can cross the bus at a time, meaning the switching speed of the router is limited by the bandwidth of the bus.

Switching via an interconnection network

- A crossbar switch is an interconnection network consisting of $2N$ buses that connect N input ports to N output ports.
- Each vertical bus intersects a horizontal bus at a crosspoint.
- Each crosspoint can be opened or closed by the switch fabric controller to connect a horizontal and vertical bus.
- Thus, **only one** crosspoint needs to be closed for an input port attached to a horizontal bus to be passed onto the vertical bus at that crosspoint (the electrical signal will always travel the full width of the bus but only be picked up by the closed crosspoint).
- Crossbar switches are non-blocking meaning so long as two packets wanting to be transmitted each belong to a different input and output port, blocking will not occur as the buses are independent of each other.
- If two input ports want to each send a packet to the same output port, this is known as [output port contention](#).

4.2.4 Queuing

- Queuing occurs both at input ports and output ports of a router.
- Memory at ports can be exhausted as queues grow larger causing packet loss.

HOL in Input Queues

- Assume we have a single FIFO input queue at a router input port.
- The first packet in the queue (head) wants to go to output port A but port A is currently unable to receive it (e.g bus might be busy).
- The second packet in the queue wants to go to output port B which is idle and ready to receive data.
- Thus, the second packet is blocked from being switched and must wait for the blockage to clear up before being able to do so.

Output Queuing

- Assume a switching fabric is N times faster than an outgoing link.
- N packets could arrive at the output port in the time it takes to transmit only 1 packet.
- Thus, a queue could grow without bound and exhaust all memory at an output port.
- When there is insufficient memory to buffer an incoming packet, a decision is made to either drop the arriving packet (known as *tail-drop*) or remove a buffered packet to make room for the new packet.
- In some cases, a packet is intentionally dropped (or the header marked with congestion bits) before the buffer is full to signal network congestion to the sender.
- A consequence of output queuing is that a *packet scheduler* at the output port must choose one queued packet for transmission.

Bufferbloat

- Bufferbloat is a phenomenon where a network link becomes extremely slow because a router between the sender and receiver has a memory buffer that is too large and always full.
- Buffers are designed to prevent packet loss during bursts however buffers that are too large can force every incoming packet to suffer a queueing delay.
- In a healthy network buffers should be empty most of the time and only fill up momentarily during a traffic spike.
- TCP is designed to regulate its transmission rate when packets are dropped.
- If a router is suffering bufferbloat, it will rarely drop packets and TCP will think that the path is clear to keep sending packets.

4.2.5 Packet Scheduling

Priority Queuing

- Priority queueing is a packet scheduling mechanism that creates multiple different FIFO queues of packets for different priority classes.
- For example, prioritisation may involve specific TCP/UDP port numbers carrying network management information, or real-time vs non-real-time.
- When choosing a packet to transmit, the scheduler will transmit a packet from the highest priority class that has a non-empty queue.

Round Robin Queuing

- Round robin queueing is much like priority queueing.
- However packet selection cycles through each of the classes, transmitting the first packet in each queue (if there is one) before repeating the cycle.

Weighted Fair Queuing (WFQ)

- WFQ is a packet scheduling mechanism commonly used in routers which is much like round robin queueing, serving classes in a circular manner.
- A router can be configured to apply different weights to different types of traffic.
- High weights mean low priority (e.g background updates), low weights mean high priority (e.g real-time packets).
- The share of the bandwidth is calculated with the following formula:

$$\text{Bandwidth Share of Queue}_i = \frac{\frac{1}{w_i}}{\sum \frac{1}{w_j}}$$

- For example, if you have two queues A with a weight of 1 and B with a weight of 2, $\text{share}_A = 0.66$ (66%) and $\text{share}_B = 0.33$ (or 33%).
- WFQ is *work-conserving* meaning if certain queues are empty, the link will not sit idly for the amount of bandwidth allocated to that queue and will instead pass it to other queues. The weight only kicks in during congestion when there are competing queues.

5 The Data Link Layer

5.1 Frames

- A frame is the data unit at layer 2.
- A frame packages bits transmitted at layer 1.

5.1.1 Structure

Preamble

- Size: 7 bytes
- Shape: 10101010
- Purpose: Allows receivers to synchronize their clock at the bit-level by detecting the signal of an incoming frame. The receiver will lock onto the signal timing to know where each bit will start and end.

Start Frame Delimiter (SFD)

- Size: 1 byte
- Shape: 10101011
- Purpose: Signals to the receiver the start of a frame. The bit at the end breaks the pattern of the preamble signalling the start of the frame data.

Destination MAC Address

- Size: 6 bytes
- Purpose: A link-layer switch reads this field to determine where to send the frame (the target device).
- Special cases:
 - Broadcast: `FF:FF:FF:FF:FF:FF` - Frame goes to all devices on the network
 - Specific range of MAC addresses - Frame goes to multiple subscribed devices

Source MAC Address

- Size: 6 bytes
- Purpose: Identifies the MAC address of the sender of the frame and lets switches learn MAC addresses to build their forwarding tables.

EtherType / Length

- Size: 2 bytes
- Shape: $\text{EtherType} \geq 0x0600$ (decimal 1536), $\text{Length} \leq 1500$
- EtherType: Specifies which layer 3 protocol is encapsulated (e.g IPv4 = `0x0800`, IPv6 = `0x86DD`, ARP = `0x0806`)
- Length: Indicates how many bytes are in the payload.
- Purpose: Lets the receiver know how to interpret the payload.

Payload / Data

- Size: 46-1500 bytes
- Purpose: Carries actual data, usually a layer 3 packet
- Minimum size is 46 bytes, if size is smaller padding bytes (0x00) are added to the end of the payload to meet the minimum ethernet frame size of 64 bytes.
- Maximum size is 1500 bytes which is the standard MTU (Maximum Transmission Unit) for ethernet.

Frame Check Sequence (FCS)

- Size: 4 bytes
- Purpose: Error detection using CRC (Cyclic Redundancy Check).
- The sender calculates a checksum over all previous bits in the frame and stores it in the FCS.
- The receiver recalculates the checksum and compares it to the FCS. If they match the frame is likely ok, otherwise it is discarded.

5.1.2 Data Loss

- Error checking occurs at the NIC (network interface card) where the FCS and MAC address are checked that they are ok.
- If error detection fails, the frame is discarded/dropped which causes data loss at layer 2.
- Data loss is handled differently depending on the upper layer (e.g TCP, lost frame = lost segment = TCP detects missing sequence numbers = TCP retransmits the segment).

5.1.3 Sending Frames

- When an application wants to send data, the device creates an IP packet with the source IP and destination IP.
- The IP layer asks "is the destination on my local network or somewhere else?".
- Every device has a routing table. If a destination matches a local route, send the packet directly otherwise send to the default gateway (a router's IP address on the local network).
- The routing table contains information about IP addresses, their gateways, subnet masks, and what outgoing network interface to use for them.

- Once the device decides where the IP packet must go using the routing table it needs a destination MAC address to build an ethernet frame.

5.1.4 Building Ethernet Frames

- Once the device knows the local IP to send to and which network interface (found in the routing table), it has to find the corresponding MAC address.
- The device checks the network interface's ARP table for the MAC address.
- If missing, an ARP request is sent/broadcast asking "who has IP x?" to obtain the MAC address.
- An ethernet frame is constructed with a destination MAC address of `FF:FF:FF:FF:FF:FF` (broadcast) and exits the device using the network interface.
- The corresponding device then responds to the request with the MAC address which is then cached in the ARP table.
- An ethernet frame can now be constructed.

5.2 Link Layer Switches

- A layer 2 switch operates on ethernet frames.
- The switch maintains a MAC address table where each entry contains a MAC address, the port and a timestamp of when it was recorded.
- MAC table entries expire after a period of inactivity (typically 300 seconds) which prevents stale entries when devices move or disconnect.
- A switch is only responsible for accepting incoming frames and forwarding them to their destination.
- MAC Learning: When a frame arrives on a port the switch updates the table with the source MAC address, port number, and timestamp or inserts into the table.

5.2.1 Forwarding

- Destination MAC is known: The switch looks up the MAC address in the table and forwards the frame only to the correct port.
- Destination MAC is unknown: The switch floods the frame out of all ports except the incoming one (unknown unicast flooding). When the destination device replies, the switch learns its MAC and future frames are sent directly.
- Destination MAC = `FF:FF:FF:FF:FF:FF`: The switch floods the frame out of all ports except the incoming one (used for things like ARP requests and DHCP discovery).

6 The Physical Layer

6.1 Access Networks

6.1.1 Digital Subscriber Line

- DSL uses existing telephone infrastructure in order to provide internet access to homes.
- A DSL modem inside a home takes digital data and translate it into high frequency tones for transmission over telephone wires to the local central office of a telco/ISP.
- The telephone line carries both telephone signals and data simultaneously.
- The signals are encoded at difference frequencies:
 - 0-4kHz band: two-way telephone channel .
 - 4-50kHz band: medium-speed upstream data channel.
 - 50kHz-1MHz: high-speed upstream data. channel
- On the customer side, a splitter splits data and telephone signals and forwards data signals to the modem.

6.1.2 Cable Internet

- Cable internet access uses television infrastructure in order to provide internet access to homes.
- Cable internet uses coaxial cables to provide internet and television signals to homes.
- It is a shared broadcast medium meaning every downstream packet travels down every downstream link to every home.
- This means that internet may slower for users when traffic is heavy and a lot of bandwidth is taken up.

6.1.3 Other Forms of Access

- FTTH (Fiber To The Home)
- Ethernet
- 5G Fixed Wireless
- WiFi

6.2 Physical Media

- Whenever a bit propagates through a network, it will travel across several physical media.
- A bit is sent by propagating electromagnetic waves or optical pulses through a physical medium between a transmitter-receiver pair.
- Physical media come in two types: guided media and unguided media.
- With guided media, the waves are guided along a solid medium (e.g copper cable, optic fibre).
- With unguided media, the waves propagate in the atmosphere or outer space (e.g wireless LAN, digital satellite channel).

Glossary

Average Throughput The total amount of data transferred divided by the total time taken for a data transfer, representing the overall efficiency of the connection. [5](#), [37](#)

Circuit Switching A network transmission method that establishes a dedicated physical or logical connection between communication devices for the duration of a session. [3](#), [4](#), [37](#)

DNS Zone A portion/division of the DNS namespace managed by a specific authoritative name server, containing the resource records for a domain and its delegated subdomains. [14](#), [15](#), [37](#)

Frequency Division Multiplexing (FDM) A multiplexing technique that partitions the total bandwidth of a link into separate frequency channels, each dedicated to a specific user or connection. [4](#), [37](#)

Head-of-Line Blocking A queueing phenomenon where the first item (the "head") in a sequence prevents subsequent items from being processed, even if those later items are ready and resources are available to handle them. This typically occurs in FIFO systems where strict ordering is enforced. [6](#), [7](#), [31](#), [37](#)

HTTP/1.1 Pipelining A technique in HTTP/1.1 that allows a client to send multiple HTTP requests over a single TCP connection without waiting for the corresponding responses, however, requiring that the server return responses in the exact order that its requests were received. [6](#), [37](#)

- Instantaneous Throughput** The rate measured in bits per second at which a destination host is receiving data at any specific point in time. 5, 37
- Iterative DNS Query** A DNS query where the receiving server returns a referral to other name servers if it does not have the specific record, leaving the client to continue the search process. 8, 37
- Output Port Contention** A traffic jam inside a router or switch that happens when multiple input ports try to send a packet to the same output port at the exact same time, forcing one of the packets to wait whilst the other is transmitted. 30, 37
- Packet Switch** A network device that moves packets from input links onto output links using a store-and-forward mechanism. 3, 6, 37
- Packet Switching** A network transmission method that breaks data up into into small chunks called packets before transmission and is then reassembled at the destination. 3, 4, 37
- Processing Delay** The time required by a router to examine a packet's header, determine its outgoing link, and place it in the link's queue. 5, 37
- Propagation Delay** The time required for a single bit to travel across a link. 5, 37
- Queuing Delay** The time a packet waits in an a router's output buffer before being transmitted onto the outgoing link. 5, 6, 31, 37
- Recursive DNS Query** A DNS query where the client requires a recursive resolver to manage the entire resolution process, returning either the final resource record or an error message. 8, 37
- Store-and-Forward** A transmission method where packet switches must receive an entire packet and process it whole before it can transmit the first bit of the packet onto the outbound link. 3, 37
- Throughput** The actual rate of successful data delivery over a communication link or network. 6, 37
- Time Division Multiplexing (TDM)** A multiplexing technique that partitions a communication link into distinct time slots. Each connection is allocated a specific, recurring, periodic slot in which to transmit data, ensuring users share the link by taking turns in the time domain. 4, 37
- Transmission Delay** The time taken to push all bits of a packet onto a link, calculated as L/R (packet length over transmission rate). 5, 37
- Transmission Rate** The hardware-defined maximum speed, measured in bits per second at which a link can transmit data. 3–6, 37